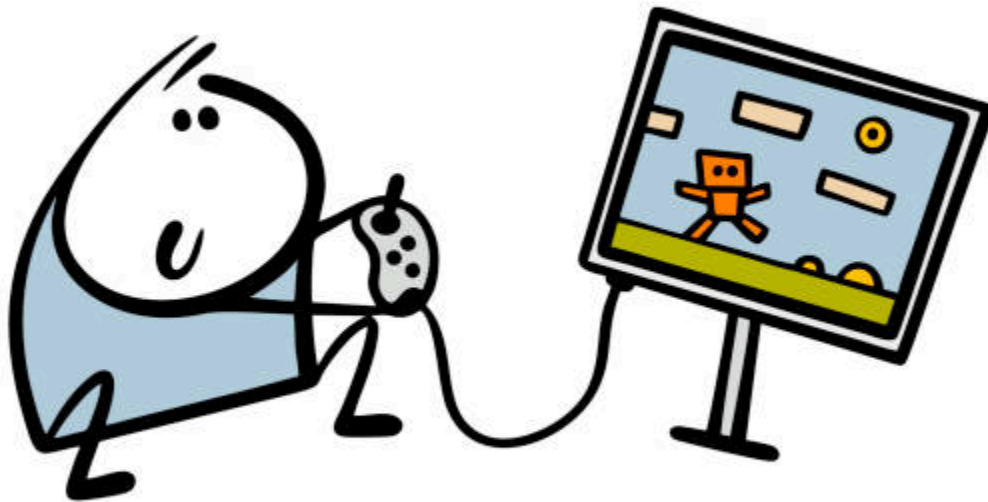


1 Title Page

Project title: RuneVault



Course: CS247 – Software Engineering Principles

Instructor: Victoria Sakhnini

Student Info:

Name: Lily Liu

Student ID: 21118218

Username (watID): l425liu

Submission date: August 5th, 11:59 PM.

Table of Contents

1 Title Page	1
Table of Contents	2
3. Design Patterns Used	3
4. Answers to Design Questions	7

[2. UML Class Diagram](#)

3. Design Patterns Used

This section is central to CS247 and should be written with care. For each design pattern you applied, provide the following four items:

- Pattern name: e.g., Observer, Factory Method, Strategy, Decorator, Template Method, MVC, etc.
- Where is it used: Which classes or components implement this pattern? Name them explicitly.
- Why you chose it: What design problem does it solve? What alternatives did you consider, and why did you reject them?
- How it is implemented: A brief description (2–4 sentences) of how the pattern manifests in your code. You may refer to your UML diagram for illustration.

1. Factory Method

Where it is used:

The Factory Method design pattern is used to create the primary game objects: Arcanist, Spectre, Item. The products created include concrete Arcanist, Spectre, and Item types as well

Motivation:

What design problems it solves:

The game involves creating many different kinds of objects whose type can only be determined at runtime based on user input and other game data. If we were to place the logic for object creation inside the GameModel we would require many switch statements or if-else blocks. Furthermore, adding more game objects would require directly modifying the GameModel which violates the open/closed principle.

Alternatives:

An alternative that was briefly considered was creating objects only where they were needed but the same issue arises of having to modify unrelated classes if we wanted to add new Arcanists, Spectres, or Items.

Implementation:

2.1 Strategy Pattern - Arcanist Ability

Where is it used:

The strategy pattern is used to determine the concrete abilities for the different Arcanists (Sage, Hexblade, Warden, Voidwalker)

Motivation:

Different Arcanist types come equipped with different characteristics such as scoring, scroll effects, damage etc. Having to scatter conditionals throughout different classes to implement these rules would render the code extremely messy.

Alternatives:

It would be possible to have subclasses override methods depending on Arcanist types but this would violate the principle of least knowledge since game logic would require knowing which specific method to call for each subclass.

Implementation:

2.2 Strategy Pattern - Spectre Hostility

Where it is used:

We use a HostilityStrategy interface to represent different types of spectre hostilities. Each Spectre contains a HostilityStrategy to determine whether it should attack an Arcanist in the given situation.

Motivation:

Different spectres may follow different hostility rules. For example, Lich hostility depends on whether or not a Lich has already been attacked, and a Vault Anchor is hostile only when the Arcanist is near the associated Shard coffer.

Alternatives:

If we had a single method in the Spectre class for determining spectre hostility this would involve many checks and become difficult to extend (violating open/closed principle). Furthermore, the Spectre base class would now be responsible for rules attaining only to enemies.

Implementation:

3. Decorator Pattern

Where is it used:

The decorator pattern is within the statistics subsystem to modify an Arcanist's combat statistics.

Motivation:

Scrolls carry effects that are able to temporarily/permanently modify the power of an Arcanist. Multiple effects can be placed on an Arcanist at one given time. Decorator patterns allow us to add,remove, and stack individual stat effects as needed without modifying the BaseStats class.

Alternatives:

Storing all possible combinations in 1 class or creating lots of subclasses would require lots of conditional logic or a lot of subclasses.

Implementation:

4. MVC (Model View Controller) Pattern

Where is it used:

The **Model** aka game logic consist of the GameModel, Board, entity classes, game objects, and all the subsystems that implement movement, spawning, attacking, and collection

The **View** aka display is implemented through TextDisplay. There is no logic in this class.

The **Controller** aka delegator will read user commands from the input stream and translate them into operations for the model. It will then request updated output to display on the View.

Motivation:

Since the game involves both a live user interface and heavy business logic, MVC was chosen such that changes to the display would not require changes to the core game logic. This separation is particularly beneficial in this use case as the game state is changing constantly based on user input thus the Model can focus primarily on maintaining the consistency of the game rules while the View handles all terminal-based concerns. The Controller acts as an intermediary and helps separate concerns.

Alternatives

Combining all the steps of a game including command parsing, board rendering, attacking, etc in only one main game class would create a very highly coupled design that is difficult to extend, and difficult to replace.

Implementation:

5. State Pattern

Where is it used:

The State pattern is implemented in the GameState interface and represents the possible states that the Game can be in.

Motivation:

User commands and actions in the game that are valid depend on the current state of the Game. For example, character selection is only valid at the start state of the game, movement is valid only when a level is active, and regular gameplay should terminate after victory/defeat. State pattern solves this issue by having one class for each possible game state such that each class handles only the commands and transitions relevant to its own phase.

Alternatives:

It is also possible to store these phases inside the GameModel with an enum and conditional statement, however this bloats the GameModel class and state-dependent conditions would be duplicated in the model class as well.

Implementation:

6. Command Pattern

Where it is used:

The Command pattern is used to represent the different actions a player can make: Move, Attack, UseScroll etc which are concrete implementations. The Controller selects and executes commands based on the player's input while the gameModel receives the operation.

Motivation:

The controller should be able to recognize player commands but not actually implement the commands themselves to enforce single responsibility principle. This action should be the responsibility of the Command class by packaging the action as its own object to reduce coupling

Alternatives:

If we didn't use the Command pattern, we could have a large chain of switch statements inside the Controller but this would cause the Controller to acquire responsibilities that implement game logic. This design would be difficult to maintain because then adding new actions would require modifying the Controller directly. The command pattern can encapsulate each request inside a separate object and include any required parameters needed.

Implementation:

4. Answers to Design Questions

Design Question 1, Arcanist Class Generation

How could you design your system so that each Arcanist class can be easily

generated, and its class ability cleanly applied without scattering conditionals throughout the codebase? How difficult does such a design make it to add a fifth class later?

Your answer should address: what class hierarchy or abstract interface you would define for the Arcanist, how class-specific behaviour (stat modifiers, ability triggers) is encapsulated, and whether your design already supports adding a new class without modifying existing code.

The Arcanist class consists of 3 main concerns.

1. The Arcanist hierarchy
2. Arcanist creation
3. Arcanist abilities

Therefore, I designed my system to use a combination of different design patterns that each target one of these concerns.

1. Arcanist Hierarchy

- The system defines an abstract base class that contains all state/behavior shared by every playable class. This includes the Arcanist's position, inventory, rune-fragment count.
- The 4 playable classes: Sage, Hexblade, Warden, VoidWalker are concrete subclasses of Arcanist that will override their corresponding methods with the own class's behaviour

2. Arcanist Creation

- A factory should be responsible for purely the creation of the Arcanist object. The system should define an ArcanistFactory method that returns an owning pointer to the abstract Arcanist type that will be decided dynamically at run time.
- The implementation of the factory will involve creating the corresponding concrete class

3. Arcanist Abilities

- Instead of implementing

Design Question 2, Generating Different Spectre Types

How does your system handle generating the different Spectre types? Is the approach different from how you generate the Arcanist class? Why or why not?

Your answer should address: what creation mechanism you use (Factory, polymorphic construction, registry), whether Arcanist and Spectre generation share a common abstraction, and the trade-offs of your chosen approach.

Design Question 3, Specter Special Abilities

How could you implement special abilities for Specters, for example, Focus Point drain for Wraiths, Ward reduction for Banshees, or rune-fragment theft for Shades? What design pattern best models abilities that modify combat outcomes without permanently altering the character objects involved?

Your answer should address: the design pattern(s) that support composable, swappable combat effects; how your design avoids combinatorial branching as new abilities are added; and whether your current implementation uses or could use this approach.

Design Question 4, Temporary Scroll Effects

What design pattern could you use to model the temporary effects of Surge Power, Fortify Ward, Sap Power, and Erode Ward, so that you do not need to explicitly track which scrolls the Arcanist has used on a given level? Consider how level transitions should trigger the reversal of these effects.

Your answer should address: the pattern that cleanly wraps and unwraps attribute modifications; how the original stats are restored at level end

without a separate tracking list; and whether your implementation already uses this approach or could be refactored to do so

Design Question 5, Unified Item Generation

How could you design item generation so that Rune Fragments, Scrolls, and Major Items reuse as much code as possible? How would you structure your hierarchy to avoid duplicating the 'guarded placement' logic shared by the Shard Coffers and the Aegis Cloak? Your answer should address: what abstract Item hierarchy you define; how a common item-generation interface can spawn any item type; and how guarded placement (Vault Anchor +item pair) is abstracted so it works for both Shard Coffers and the Aegis Cloak without duplication